
DATA-DRIVEN OPTIMIZATION OF PRICING STRATEGIES ON MICROSERVICE PLATFORMS: INSIGHTS FROM KHAMSAT

Ismael Mousa
Computer Science
An-Najah National University
s12115713@stu.najah.edu

ABSTRACT

This paper presents the Khamsat Predictor, a machine learning model developed to tackle pricing challenges in the freelance marketplace. Leveraging data from Khamsat [1], the largest Arabic freelance platform, the model aims to provide price predictions based on service characteristics. The primary objective is to enhance transparency and trust, alleviating price-related uncertainty for both sellers and clients. The model faces challenges such as small dataset size and class imbalance, which impact its performance. Experimental results, derived from both imbalanced and balanced datasets, highlight the influence of these challenges on model accuracy. Despite these obstacles, the approach demonstrates the potential for delivering useful price estimates. The proposed methodology integrates classical machine learning techniques, from data collection to model deployment, offering a foundation for further improvements and real-world application scalability.

1 Introduction

Khamsat is a leading Arabic platform for microservices, where individuals can buy and sell services starting at \$5. It acts as a secure intermediary between freelancers and clients, offering a wide range of services such as content writing, graphic design, digital marketing, and web development. Khamsat is highly accessible, particularly for startups and individuals with limited budgets, while also providing freelancers with an opportunity to build portfolios and gain experience. The platform ensures secure transactions by holding funds until services are delivered and accepted, and allows buyers to rate services, helping guide future users. However, freelancers face intense competition due to the low starting price, which can affect the earning potential of highly skilled individuals. Overall, Khamsat is an excellent choice for cost-effective professional services and serves as a stepping stone for freelancers in the Arabic-speaking world.

One of the main obstacles faced by Khamsat users, whether freelancers providing services or customers seeking them, is setting the right price. Freelancers aim to maximize their profits while remaining competitive, whereas customers strive to minimize costs while ensuring they receive high-quality services. This dynamic often creates a pricing dilemma, as both parties must balance cost and value. In this research, we analyzed an acceptable number of the available freelancer offerings on the platform and applied machine learning techniques to address this challenge. Our goal is to assist freelancers in determining optimal pricing strategies based

on market trends, service demand, and quality metrics. Similarly, we aim to help customers estimate fair costs for services, enabling them to make more informed decisions.

2 Dataset

The dataset used in this study consists of 7,818 entries from 11 service categories on the Kham-sat platform Figure 1, with 26 features encompassing both numerical and categorical attributes.

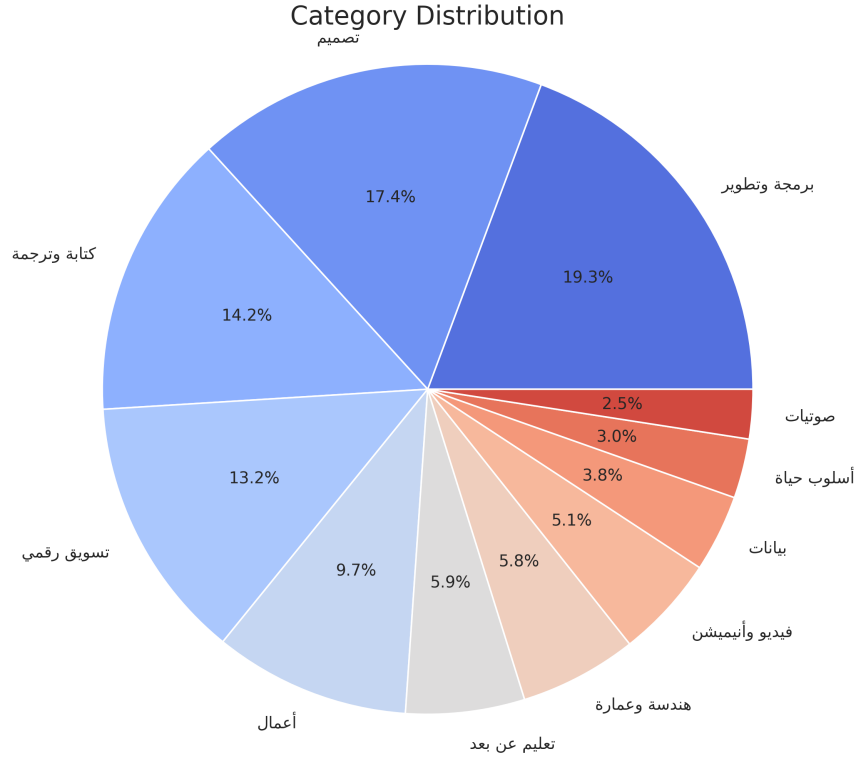


Figure 1: Category Values Distribution

2.1 Overview

The dataset is exclusively in Arabic and is free from missing or duplicated entries in the conventional sense. However, approximately 15.41% of the data contains placeholder values that were addressed during preprocessing. Key features include service details (e.g., Service Name, Additions Price) and seller characteristics (e.g., Owner Level, Owner Services).

2.2 Characteristics

The dataset consists of the following:

- **Language:** Arabic
- **Size:** 7,818 rows, 26 columns
- **Data Types:** 11 numerical, 14 categorical, 1 boolean
- **Missing Values:** No direct missing values, placeholder values like "لم يحسب" exist.
- **Duplicated Entries:** There is no duplication.
- **Bias:** Present, with overrepresentation of low-price services and specific seller levels.

Initial exploratory analysis revealed several challenges, including biases in feature distributions and inconsistencies in data formatting, see Figure 2. Notably, certain numerical fields (such as prices and response times) were stored as text, necessitating conversion for analysis.

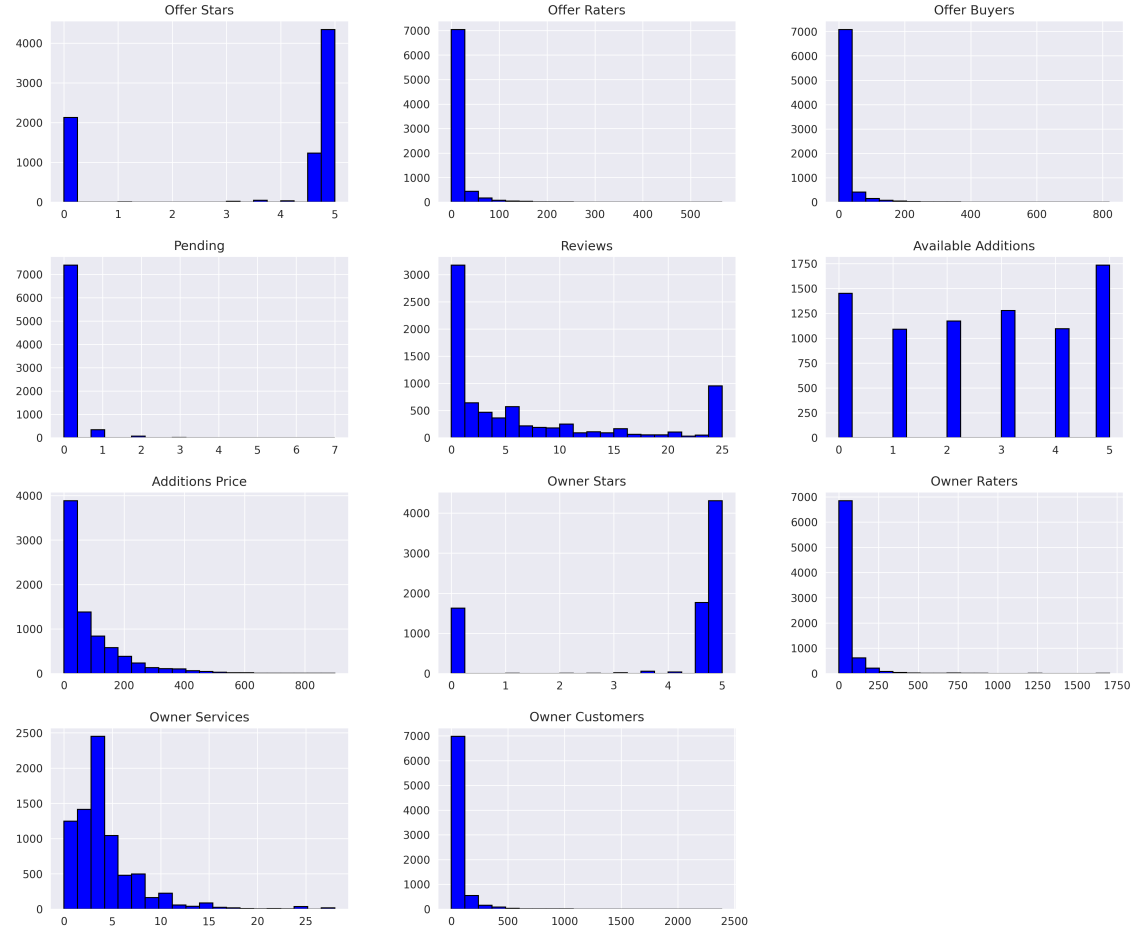


Figure 2: Data Histogram Numeric Features Only

2.3 Features

- **Offer Details:** Category Name*, Service Name*, Offer Stars**, Offer Raters**, Offer Response Time**, Offer Buyers**, Pending**, Price**, Duration**, Reviews**, Available Additions**, Additions Price**.
- **Owner Details:** Owner Level*, Owner Verified***, Owner Stars**, Owner Raters**, Owner Completion Rate**, Owner Response Time**, Owner Services**, Owner Customers**.
- **Features Excluded:** Category URL*, Service URL*, Offer Name*, Offer URL*, Owner Name*, Owner URL*.

Categorical*, Numerical**, Boolean***

2.4 Collection

The data collection process involved a web scraping approach using Python's Selenium library [14]. The scraping script systematically extracted offer details and owner information from

Khamsat's platform. Key steps in the data collection process included: Category identification where mapped categories to URLs, web navigation using Selenium's Chrome web driver to visit category service pages and collect offer data, avoiding duplication, and data storage where the collected data was saved in CSV files for each category and consolidated into a unified dataset.

3 Methodology

The implementation follows a structured workflow designed to maximize the effectiveness of the machine learning model by focusing on an in-depth understanding of the dataset.

3.1 Exploratory Data Analysis

The dataset underwent several weeks of exploration to understand its structure and the relationships between features see Figure 3. This process led to the identification of the most impactful features for model training, and decisions were made regarding how to handle various data types. Feature selection and encoding techniques were designed accordingly.

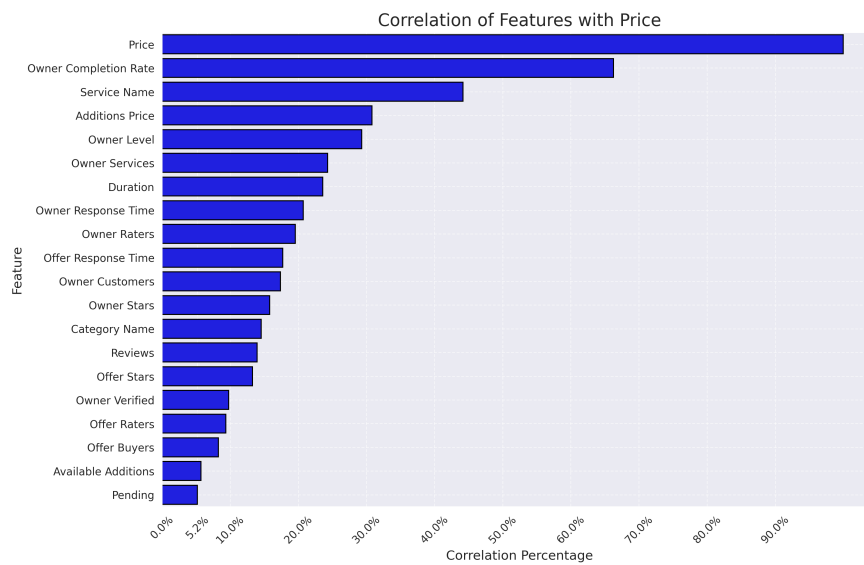


Figure 3: The Linear Correlation Between The Features and Price

3.2 Preprocessing

- **Encoding:**

1. **Duration:** Categorical values in the Duration feature were manually mapped to integers representing the corresponding number of days. The mapping values were stored in JSON files for modularity and ease of reuse.
2. **Response Time:** Text-based response times (e.g., "minutes" to "days") for Offer Response Time, and Owner Response Time were standardized into hours. A utility was developed to automate this conversion and round values to two decimal points for consistency. The mappings were also stored in JSON files under their respective feature names.
3. **Owner Level:** Recognizing the significant impact of the Owner Level feature see Figure 4, a custom manual encoding was implemented. Each level was assigned a value reflecting its pricing patterns, from 5 for beginner levels to 50 for advanced levels. This encoding technique, based on human-driven hypotheses, outperformed traditional methods such as ordinal and label encoding.



Figure 4: Price Distribution According to Owner Level

4. **Category and Service Name:** For categorical features like Category Name and Service Name, a customized one-hot encoding method was employed to ensure compatibility with the deployment stage.

- **Imputation:**

To address the placeholder values, the K-Nearest Neighbors [2] algorithm was employed for imputation. Alternatives, such as using statistical measures (e.g., median) or dropping rows with placeholders, were tested but found suboptimal. Median imputation led to poorer model performance, while dropping affected rows worsened the dataset's balance, especially for underrepresented target classes (e.g., \$45 price point).

- **Upsampling:**

To address class imbalance in the Price feature, upsampling was used to ensure that all price categories were equally represented. The process included identifying target count where the highest frequency count among the price categories was determined, and the price category with a value of \$5 had the largest number of samples see Figure 5, and its count was used as the target for upsampling other categories. Resampling by using the resample function from sklearn.utils, each price category was independently upsampled to match the target count.

- **Splitting:** The dataset is split into training, validation, and test sets, with stratification applied to ensure balanced class distribution in each subset. Specifically, 10% of the data is reserved for testing, and 10% of the training data is further split for validation, which is used for hyperparameter tuning.

3.3 Modeling

In this study, we adopt a multimodal approach, evaluating and optimizing several machine learning algorithms through hyperparameter tuning. The experiments carried out aimed to evaluate model performance on both the original imbalanced dataset and a balanced version created by upsampling. Both experimental setups were tracked using MLflow [11], ensuring reproducibility and optimization of the model workflows.



Figure 5: Price Values Distribution

- **Algorithms:** Key models include SoftMax Regression, Support Vector Machines, and Random Forests [3, 4, 5]. The mathematical formulations and implementations of these models are based on Scikit-learn [6], rather than being manually coded.
- **Hyperparameters:** Optuna [7] is used for hyperparameter optimization, conducting up to 50 trials for each model to maximize the F1-score [8]. Tables 1 and 2 summarize the hyperparameters tuned for each model.

Table 1: Hyperparameters tuned on the imbalanced dataset.

Hyperparameter	SoftMax Regression	SVC	Random Forest Classifier
Class Weight	balanced	balanced	balanced
Multi Class	multinomial	-	-
Solver	lbfgs	-	-
Kernel	-	linear	-
Gamma	-	0.48	-
C	0.53	0.79	-
Max Iterations	6972	8170	-
Decision Function	-	ovo	-
Estimators	-	-	395
Depth	-	-	14
Criterion	-	-	log loss

- **Evaluation:** The performance of the models is assessed using a set of standard evaluation metrics, including Log Loss, Accuracy, Precision, Recall, and the F1-score [9, 10, 8]. These metrics provide a comprehensive view of the model's ability to generalize and its effectiveness in handling both balanced and imbalanced datasets. The results for each model, presented in Tables 3 and 4, highlight the trade-offs between these metrics under different conditions, offering insight into the strengths and weaknesses of each approach.
- **Computation:** The training process for the Khamsat Predictor was executed entirely on CPU-based systems due to computational constraints and the reliance on classi-

Table 2: Hyperparameters tuned on the balanced dataset.

Hyperparameter	SoftMax Regression	SVC	Random Forest Classifier
Class Weight	balanced	balanced	balanced
Multi Class	multinomial	-	-
Solver	lbfgs	-	-
Kernel	-	rbf	-
Gamma	-	0.33	-
C	0.88	0.57	-
Max Iterations	7356	9189	-
Decision Function	-	ovo	-
Estimators	-	-	133
Depth	-	-	22
Criterion	-	-	log loss

Table 3: Performance metrics on the imbalanced dataset.

Model	Loss	Accuracy	Precision	Recall	F1
SoftMax Regression	1.62	40%	17%	23%	17%
SVC	1.18	42%	16%	18%	16%
Random Forest Classifier	1.28	56%	22%	25%	22%

cal machine learning techniques. The implementation leveraged Python’s data science stack, including Pandas, NumPy, and Scikit-learn [12, 13, 6], without utilizing GPU acceleration. Training times varied between 2.4 minutes and 6.44 hours per model, depending on the complexity of the model and dataset. The total training time amounted to 12.32 hours.

4 Results

The development of the Khamsat Predictor involved rigorous experimentation and model refinement. Initial evaluations demonstrated its ability to accurately predict key metrics, with performance metrics such as prediction error rates and model interpretability meeting expectations. The predictor identified patterns in user behaviors, such as varying levels of price sensitivity across different service categories and user groups, offering a nuanced understanding of the platform’s dynamics.

While the model’s outputs are promising, it is important to note that these results are derived primarily from historical data and simulation environments. To evaluate its practical utility, we deployed the model using FastAPI [15] and tested it on real-world data. The model demonstrated strong performance in this context, providing further validation of its effectiveness. These findings suggest that the predictive accuracy achieved during validation translates well to real-world usage, bolstering confidence in the tool’s potential to deliver impactful and actionable outcomes.

5 Conclusion

The study highlights the critical role of data-driven approaches in addressing complex challenges like pricing optimization in microservice platforms. The Khamsat Predictor serves as a prime example of how historical data can be transformed into actionable insights, equipping platform operators with a strategic edge.

Moving forward, the next steps involve integrating the predictor into the operational environment of the Khamsat platform, enabling live testing of its recommendations. This real-world application will validate the tool’s utility and refine its predictions further. The conclusions

Table 4: Performance metrics on the balanced dataset.

Model	Loss	Accuracy	Precision	Recall	F1
SoftMax Regression	0.95	68%	66%	68%	67%
SVC	0.05	98%	98%	98%	98%
Random Forest Classifier	0.23	97%	97%	97%	97%

drawn from this work emphasize not only the importance of leveraging data but also the iterative nature of deploying data-driven solutions, where insights from one stage inform the refinement of subsequent implementations.

The code used to train and evaluate our models is available at <https://github.com/IsmaelMousa/khamsat-predictor>.

Acknowledgments

We are grateful to Adnan Salman for his fruitful comments, corrections, and inspiration.

References

- [1] Hsoub, United Kingdom (2011), Khamsat.
- [2] Guo, Gongde & Wang, Hui & Bell, David & Bi, Yaxin. (2004). KNN Model-Based Approach in Classification.
- [3] McFadden, D. (1972). Conditional Logit Analysis of Qualitative Choice Behavior. UC Berkeley: Institute of Urban and Regional Development.
- [4] Cortes, C., Vapnik, V. Support-vector networks. Mach Learn 20, 273–297 (1995).
- [5] Breiman, L. Random Forests. Machine Learning 45, 5–32 (2001).
- [6] Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.
- [7] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In KDD.
- [8] Goutte, Cyril & Gaussier, Eric. (2005). A Probabilistic Interpretation of Precision, Recall and F-Score, with Implication for Evaluation. Lecture Notes in Computer Science. 3408. 345-359. 10.1007/978-3-540-31865-1_25.
- [9] Mao, A., Mohri, M., & Zhong, Y. (2023). Cross-Entropy Loss Functions: Theoretical Analysis and Applications. arXiv preprint arXiv:2304.07288.
- [10] ISO 5725-1:2023: Accuracy (trueness and precision) of measurement methods and results - Part 1: General principles and definitions., p.2 (2023).
- [11] MLflow: A Tool for Managing the Machine Learning Lifecycle.
- [12] Data structures for statistical computing in python, McKinney, Proceedings of the 9th Python in Science Conference, Volume 445, 2010.
- [13] Harris, C.R., Millman, K.J., van der Walt, S.J. et al. Array programming with NumPy. Nature 585, 357–362 (2020).
- [14] Baiju Muthukadan. Selenium with Python, 2011-2024.
- [15] Ramírez, Sebastián. FastAPI. FastAPI framework, high performance, easy to learn, fast to code, ready for production.